AUTOM-99: Best Practice Summary

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 04/25/2026

This notebook consolidates every actionable best practice from the AUTOM series (notebooks 01-08) into a single reference. Each practice is definitive: it tells you exactly what to set, not what to consider.

Use this as a checklist when designing, implementing, or auditing Dynatrace configuration automation.

Table of Contents

Prerequisites

Requirement	Details
AUTOM Series	Familiarity with AUTOM-01 through AUTOM-08
Dynatrace Environment	SaaS tenant with admin access
Automation Tooling	Monaco CLI, Terraform CLI, or both installed

---

1. Authentication & Token Management

Practice	Recommended Setting/Value	Priority
Use least-privilege scopes	Grant only `settings.read`, `settings.write`, `ReadConfig`, `WriteConfig` for config tools. Add `ExternalSyntheticIntegration` only when managing synthetics.	Critical
Use Platform Token + API Token together	Platform Token handles Settings 2.0 and Gen3 resources; API Token (`dt0c01.*`) handles Synthetics and SLOs. Configure both in provider.	Critical
Use OAuth clients for Gen3 platform resources	Set `client_id`, `client_secret`, `account_id` for Workflows, Documents, Segments, and IAM resources.	Critical
Never hardcode tokens	Store tokens in environment variables, HashiCorp Vault, AWS Secrets Manager, or equivalent secret store.	Critical
Separate tokens per environment	Create distinct tokens for dev, staging, and production tenants. Never share tokens across environments.	Critical
Rotate tokens on a schedule	Set a rotation cadence (e.g., 90 days). For short-lived tokens, retrieve from Vault at pipeline runtime.	Recommended

Practice	Recommended Setting/Value	Priority
Prefer Platform Tokens over classic Access Tokens	Platform Tokens scope to the user's existing permissions and simplify token management.	Recommended
Use OAuth for CI/CD service accounts	OAuth clients operate with independent scopes, making them suitable for pipelines without tying to a human user.	Recommended

2. Settings API

Practice	Recommended Setting/Value	Priority
Use Settings 2.0 API for all new configuration	Endpoint: `/api/v2/settings/objects`. Do not use Classic Config API for new configs.	Critical
Discover schemas before creating objects	Call `GET /api/v2/settings/schemas` to find the correct `schemaId`, then `GET /api/v2/settings/schemas/{schemaId}` for field requirements.	Critical
Use bulk POST for multiple objects	Send an array of objects in a single `POST /api/v2/settings/objects` call instead of individual requests.	Recommended
Check before create (idempotency)	Query for existing objects by `schemaIds` and `scopes` before creating. Use `PUT` for updates (idempotent), not `POST`.	Recommended
Implement exponential backoff on 429	Retry with increasing delay when rate-limited. Stay under 100 requests/minute for bulk operations.	Recommended
Track object IDs for lifecycle management	Store returned `objectId` values for subsequent update and delete operations.	Recommended

Practice	Recommended Setting/Value	Priority
Review existing objects as examples	Before writing new config, `GET /api/v2/settings/objects?schemaIds=<schema>` to see the structure of existing objects in your tenant.	Optional

3. Monaco Configuration-as-Code

Practice	Recommended Setting/Value	Priority
Always run `monaco validate` before deploy	Run `monaco validate manifest.yaml` to catch YAML syntax and schema errors before touching the tenant.	Critical
Always run `monaco deploy --dry-run` before apply	Preview what will change. Never deploy blind.	Critical
Use environment variables for auth	Set `DT_TENANT_URL` and `DT_API_TOKEN` as env vars. Never put tokens in YAML files.	Critical
Use meaningful config IDs	IDs should describe the config purpose (e.g., `production-mz`, `web-app-alerting`). These IDs are how Monaco tracks objects.	Critical
Use `manifest.yaml` for multi-environment setup	Define all environments in `environmentGroups` with separate URL/token env vars per environment.	Recommended
Use environment-specific overrides	Use `overrides` blocks in config.yaml to vary values (e.g., `delay_minutes: 1` for prod, `5` for staging).	Recommended
Use `skip.environments` for env-specific configs	Skip production-only configs in dev/staging using the `skip` directive.	Recommended
Use groups for related configs	Group related configs (e.g., `group: web-application`) and deploy with `--group` flag.	Recommended

Practice	Recommended Setting/Value	Priority
Use config dependencies for ordering	Reference other configs with `{{ .management-zones.production-mz.id }}` to ensure correct deployment order.	Recommended
Modular directory structure	Separate configs by domain: `management-zones/`, `auto-tagging/`, `alerting-profiles/`.	Recommended
Download before creating from scratch	Run `monaco download` to get the current state, then modify the exported YAML.	Optional

---

4. Terraform Infrastructure-as-Code

Practice	Recommended Setting/Value	Priority
Pin the provider version	`version = "~> 1.91"` in `required_providers` block. Never use unpinned versions.	Critical
Use remote state storage	Configure `backend "s3"` or `terraform.cloud` block. Never use local state for teams.	Critical
Enable state locking	Use S3+DynamoDB, Terraform Cloud, or equivalent to prevent concurrent applies.	Critical
Always run `terraform plan` before `terraform apply`	Review the plan output for every change. Automate plan-as-PR-comment in CI.	Critical
Use dual auth for full resource coverage	Configure both `dt_api_token` (for Synthetics/SLOs) and `client_id` / `client_secret` (for Gen3/IAM). The provider routes each resource to the correct auth method.	Critical
Use modules for reusable patterns	Create modules for standard patterns (e.g., `modules/environment/`, `modules/alerting-profile/`).	Recommended

Practice	Recommended Setting/Value	Priority
Use variable validation in modules	Add `validation` blocks to enforce naming, environment values, and delay constraints at module input.	Recommended
Use workspaces for multi-environment	Create `development`, `staging`, `production` workspaces. Reference `terraform.workspace` in locals for env-specific values.	Recommended
Import existing resources into state	Write the HCL block first, then `terraform import <resource> "<object-id>"`. Never recreate what already exists.	Recommended
Use `terraform state rm` to detach without deleting	When you need to stop managing a resource without destroying it.	Recommended
Format code with `terraform fmt`	Run before every commit for consistent HCL formatting.	Recommended
Use the export utility for brownfield adoption	Run `terraform-provider-dynatrace -export -id all -target-folder ./exported` to generate HCL from existing configs.	Optional

---

5. Dynatrace Workflows

Practice	Recommended Setting/Value	Priority
One workflow, one purpose	Each workflow handles a single responsibility (e.g., "problem email notification" not "do everything on problem").	Critical
Make tasks idempotent	Tasks must be safe to run multiple times without side effects (e.g., check if ticket exists before creating).	Critical
Store secrets in Credential Vault	Use Dynatrace Credential Vault for API keys, webhook URLs, and integration tokens. Never hardcode secrets in workflow YAML or JavaScript.	Critical

Practice	Recommended Setting/Value	Priority
Scope triggers narrowly	Filter triggers by management zone, entity tag, problem category, and severity. Do not trigger on all problems.	Critical
Add retry with backoff on external calls	Set `retry.count: 3` and `retry.delay: 30s` on HTTP tasks and external integrations.	Recommended
Route errors to a handler task	Define `on_error: error_handler` tasks that send Slack/Teams notifications on failure.	Recommended
Validate entity type before remediation	Check `entity.type` in a JavaScript task before executing remediation (e.g., only restart `PROCESS_GROUP_INSTANCE`, not hosts).	Recommended
Debounce flapping alerts	Add delays or deduplication checks for alerts that open/close rapidly.	Recommended
Use manual triggers for testing	Test workflow logic with manual triggers before enabling detected-problem triggers.	Recommended
Log all remediation actions	Ensure every auto-remediation action writes to the Dynatrace audit trail and comments on the originating problem.	Recommended
Use sub-workflows for reusable logic	Extract common patterns (e.g., "create ITSM ticket") into sub-workflows callable from multiple parent workflows.	Optional
Use approval gates for risky remediations	Enable built-in Approval Requests for actions like scaling, restarts, or config changes.	Optional

---

6. SDKs & Programmatic Access

Practice	Recommended Setting/Value	Priority
Use environment variables for credentials	Set `DT_URL` and `DT_API_TOKEN` as environment variables. Never pass tokens as function arguments or config literals.	Critical
Handle paginated responses	Always iterate through all pages. Check for `nextPageKey` in responses and loop until exhausted.	Critical
Implement rate limiting	Use a rate-limiting decorator/wrapper (e.g., max 50 requests/second). Respect `429` responses with exponential backoff.	Recommended
Use type-safe SDK clients	Use `@dynatrace-sdk/client-query` for TypeScript, `dt-sdk` for Python. These provide typed responses and auto-generated API coverage.	Recommended
Wrap API calls in error handlers	Catch `ApiError` specifically; log `status` and `message`. Re-raise unknown errors.	Recommended
Add structured logging	Log request method, endpoint, status code, and duration for every API call.	Recommended
Use MCP Server for AI-assisted access	Connect Dynatrace MCP Server to Claude Code, GitHub Copilot, or Amazon Q for natural-language queries against your tenant.	Optional

7. CI/CD & GitOps

Practice	Recommended Setting/Value	Priority
Store all config in Git	Every Dynatrace configuration (Monaco YAML, Terraform HCL) lives in version control. No exceptions.	Critical

Practice	Recommended Setting/Value	Priority
Require PR reviews for production changes	Enable branch protection on `main`. Require at least 1 approval. Use CODEOWNERS for Dynatrace config paths.	Critical
Mask all tokens in CI/CD	Store tokens as masked secrets in GitHub Actions, GitLab CI, or equivalent. Never echo tokens in logs.	Critical
Validate on every PR	Run `monaco validate` or `terraform validate` + `terraform plan` on every pull request. Post plan output as PR comment.	Critical
Deploy only from main branch	Gate `terraform apply` and `monaco deploy` to run only on push to `main` (not on PR branches).	Critical
Use staged rollout: dev, staging, production	Deploy to dev first, then staging, then production. Require manual approval gate before production.	Critical
Schedule drift detection	Run `terraform plan -detailed-exitcode` on a cron schedule (e.g., weekdays 6am UTC). Exit code `2` means drift. Auto-create GitHub Issue on drift.	Recommended
Use Vault for runtime credentials	Retrieve tokens at pipeline runtime from HashiCorp Vault using OIDC/JWT auth instead of static CI/CD secrets.	Recommended
Use reusable workflows for multi-repo orgs	Define a shared `workflow_call` workflow that all team repos invoke. Standardizes plan/apply across the organization.	Recommended
Send deployment events to Dynatrace	`POST /api/v2/events/ingest` with `eventType: CUSTOM_DEPLOYMENT`, commit SHA, and branch name after every deploy.	Recommended
Use Kustomize overlays for Operator GitOps	Base DynaKube config + per-cluster patches via `patchesStrategicMerge`. Use `apiVersion: dynatrace.com/v1beta5` or `v1beta6`.	Recommended

Practice	Recommended Setting/Value	Priority
Encrypt K8s secrets in Git	Use Sealed Secrets, SOPS, or External Secrets Operator. Never commit plaintext Dynatrace tokens to Git.	Critical
Use a PR template for config changes	Include sections: Description, Type of Change, Environments Affected, Validation Checklist, Rollback Plan.	Optional

---

8. Governance Architecture

Practice	Recommended Setting/Value	Priority
Separate IAM pipeline from config pipeline	Pipeline A (central team) manages `dynatrace_iam_*` resources. Pipeline B (LOB teams) manages configs under grants from Pipeline A.	Critical
Single service account writer in production	Only one SA writes to prod. All humans get read-only. Teams develop in dev/test tenant with UI write access.	Critical
Never mix Monaco and Terraform for the same config type	Choose one tool per configuration domain. Mixing causes state conflicts and unpredictable drift.	Critical
Eliminate manual changes alongside automation	If a config is managed by automation, all changes go through the pipeline. No ClickOps in production.	Critical
Always use version control for configs	Every automation artifact (YAML, HCL, JSON) lives in Git with meaningful commit messages.	Critical
Use OPA/Conftest for resource type allowlists	Define a Rego policy that restricts which `dynatrace_*` resource types each team repo can create.	Recommended
Enforce mandatory team tagging via policy	OPA/Sentinel policy requires every resource to include team ownership metadata (e.g., tag, naming prefix, MZ binding).	Recommended

Practice	Recommended Setting/Value	Priority
Lock down Terraform state file access	Restrict HCP Terraform workspace permissions. State files can contain OAuth credentials and IAM binding details.	Recommended
Use brokered self-service for Synthetic monitors	Teams submit declarative requests; a central pipeline owns the environment-wide API token and applies on their behalf. Teams never hold direct API credentials.	Recommended
Use IAM policies for schema-level access	Manage `dynatrace_iam_policy` resources with `statement_query` restricting teams to specific `settings:objects:*` schemas.	Recommended
Document v1 API limitations for auditors	Frame Synthetic v1 API scoping gap as an accepted platform constraint with compensating controls (brokered access, MZ fencing, Sentinel).	Optional
Use module allowlists in Sentinel	Restrict Terraform plans to only approved modules, preventing teams from using unapproved resource patterns.	Optional

9. Migration Automation

Practice	Recommended Setting/Value	Priority
Export before migrating	Run `monaco download` or `terraform-provider-dynatrace -export` on the source tenant. Keep the full backup.	Critical
Replace hardcoded entity IDs with selectors	Dashboards and alerting profiles contain entity IDs. Replace with entity selectors (e.g., `type(SERVICE),tag(environment:production)`) before deploying to target.	Critical
Validate counts after each migration step	Use the Settings API: `GET /api/v2/settings/objects?schemaIds=<schema>` to compare object counts between source and target tenants. Note: `fetch dt.settings` is not valid DQL.	Critical

Practice	Recommended Setting/Value	Priority
Test migration in a non-production tenant first	Deploy exported config to a dev/test tenant before touching production.	Critical
Re-enter credentials manually	Credential Vault entries, API tokens, SSL certificates, and cloud integration secrets do not migrate. Re-create them in the target.	Critical
Use Monaco for SaaS-to-SaaS migration	Download from source, edit manifests to point at target, validate, dry-run, deploy.	Recommended
Use SaaS Upgrade Assistant for Managed-to-SaaS	Access via Apps in your Dynatrace account. Provides automated discovery, compatibility checks, and progress tracking.	Recommended
Document non-portable configurations	Keep a checklist of configs that need manual re-creation: credentials, historic data, private locations, cloud integrations.	Recommended
Use Terraform export for brownfield imports	Export existing configs to HCL, then manage via Terraform going forward.	Optional

10. Tool Selection

Use Case	Use This Tool	Priority
One-off configuration change	Settings API (direct REST)	Critical
Repeatable config deployments with GitOps	Monaco	Critical
Full IaC with state management and drift detection	Terraform	Critical

Use Case	Use This Tool	Priority
Event-driven automation and auto-remediation	Dynatrace Workflows	Critical
Custom reporting, complex logic, or integrations	SDK (TypeScript or Python)	Critical
Tenant-to-tenant migration	Monaco (download/deploy pattern)	Critical
Managed-to-SaaS migration	SaaS Upgrade Assistant	Recommended
AI-assisted querying and triage	Dynatrace MCP Server	Optional
Team already uses Terraform for infrastructure	Terraform (extend existing IaC)	Recommended
Team needs lowest barrier to entry	Monaco (YAML, simple CLI)	Recommended

Anti-Patterns

Anti-Pattern	Consequence	Correct Approach
Mixing Monaco and Terraform for the same config type	State conflicts, unpredictable drift	Choose one tool per config domain
Manual UI changes alongside automation	Configuration drift, lost changes	All changes go through the pipeline
No version control for configs	No rollback, no audit trail	Store everything in Git
Hardcoded secrets in config files	Security breach risk	Use env vars, Vault, or secret managers
Distributing environment-wide write tokens to teams	No access isolation	Use brokered self-service or scoped OAuth
Using `fetch dt.metrics` in DQL	Invalid command	Use `timeseries` for metrics

---

11. Community Resources

Key GitHub repositories organized by tool, providing starter templates, working examples, and hands-on exercises.

Monaco Repositories

Repository	Description
dynatrace-configuration-as-code	Official Monaco CLI (v2.28.5)
dynatrace-configuration-as-code-samples	9 Monaco starter templates, pipeline observability configs, CI validation scripts
easytrade	Real-world Monaco project structure (manifest.yaml, detection rules, workflows)
Dynatrace-Config-Manager	GUI tool for tenant-to-tenant config migration
monaco-self-paced-exercises	6 structured hands-on exercises

Terraform Repositories

Repository	Description
terraform-provider-dynatrace	Official provider (v1.93.0, 180 releases) with export capability
dynatrace-configuration-as-code-samples	10 Terraform starter templates, reusable modules, DQL data source, IAM onboarding

CI/CD & Platform Engineering

Repository	Description
dynatrace-automation-tools	SRG + Events CLI for CI/CD pipelines (v1.0.3)
platform-engineering-demo	Full IDP reference: ArgoCD + Backstage + Keptn + Dynatrace

Repository	Description
demo-crossplane	Crossplane + Terraform GitOps pattern
monaco-demo	Working GitHub Actions workflow for Monaco deploy
obslab-release-validation	Release validation with k6, business events, and SRG

> **Migration note:** The `Dynatrace/community-examples` repository has an empty `configuration-as-code/` directory with a note that CaC samples will migrate from `dynatrace-configuration-as-code-samples` in a future update. Reference both repos until the migration completes.
